

APACHE DORIS 3.0.8

复习题答案

第七章：数据导入与实时写入

20 道综合题参考答案。每题包含一句话结论、底层因果链、得分关键词与常见误区。

配套资料：notes-07

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航与评分

建议先口述再核对。每题 10 分：结论 2 分，机制 4 分，因果链 2 分，边界或证据 2 分。

题号	主题	关键词
1-6	导入链路 with 批量文件	事务、Push/Pull、并行度
7-11	Routine Load	Job/Task、Offset、微批、状态
12-14	Flink 与 2PC	Stream Load、Checkpoint、Sequence
15-18	小批与一致性	Group Commit、质量、Label、幂等
19-20	存储与诊断	Rowset、Compaction、因果链

高质量回答结构



1. 为什么 Doris 导入不是简单地把文件复制到 BE ？

因为外部字节必须先被转换为符合表模型、分区分桶和列式存储规则的新事务版本。

Doris 要解析格式、映射字段、执行类型转换和质量检查，应用 Duplicate/Unique/Aggregate 语义，再计算目标 Tablet，写入多副本并 Commit/Publish。最终产物是 Rowset/Segment 和版本元数据，而不是原 CSV 文件。

得分关键词

解析；转换；表模型；Tablet 路由；副本；事务版本

常见误区：只描述成上传文件，忽略表模型和事务发布；这无法解释半批次为何不可见。

2. FE 和 BE 在导入链路中分别负责什么？

FE 是控制面，负责认证、元数据、事务与发布；BE 是数据面，负责接收、解析、路由和落盘。

FE 决定怎样写但不承载主要数据流，避免成为吞吐瓶颈。BE 利用分布式并行完成格式处理、MemTable、Rowset、Segment 和副本写入。事务结果再由 FE 统一提交和发布。

得分关键词

控制面；数据面；事务；元数据；MemTable；Rowset

常见误区：认为所有 Stream Load 数据先完整经过 FE；正常链路会重定向到协调 BE。

3. Stream Load 与 Broker Load 的数据流方向有什么不同？

Stream Load 由客户端通过 HTTP Push；Broker Load 由 Doris 根据远程路径从 S3/HDFS Pull。

前者适合本地文件和应用批次，客户端负责发起、发送与重试；后者只提交 Load Job，FE 枚举文件并调度多个 BE 并行读取，适合共享存储中的大规模文件。

得分关键词

Push；Pull；HTTP；远程存储；异步 Job

常见误区：把两者只区分为同步和异步，而没有说明数据由谁主动传输。

4. 为什么 Broker Load 更适合 S3/HDFS 上的大规模历史回灌？

因为文件已经在可并行读取的共享存储中，Doris 可以把读取和导入任务分散到多个 BE，而不依赖单客户端二次上传。

FE 根据文件、可切分性和集群资源生成任务，多个 BE 分别拉取、解析并写目标 Tablet。这样能利用存储和 Doris 两侧并行，并通过 Label 与 SHOW LOAD 管理长任务。

得分关键词

共享存储；分布式读取；异步；Label；SHOW LOAD

常见误区：认为一个超大不可切分 gzip 也能自动获得任意并行度；文件布局仍会限制吞吐。

5. Broker Load 和 INSERT INTO SELECT + TVF 应怎样选择？

大规模、异步、需要 Load Job 管理的回灌优先 Broker Load；需要预览和复杂 SQL 转换时优先 TVF + INSERT SELECT。

Broker Load 提供 Label、状态与多数据描述事务；TVF 把外部文件当表，可先 SELECT，再使用 JOIN、CAST、函数和完整 SQL 表达式。选择应看数据量、转换复杂度、同步返回和原子边界。

得分关键词

异步回灌；TVF；预览；复杂 SELECT；多表原子

常见误区：只按文件格式选择。两者都可读取 Parquet，关键是作业和转换语义。

6. Broker Load 的并行度受到哪些因素限制？

并行度由文件数量、文件可切分性、可用 BE、远端吞吐和目标 Tablet 分布共同决定。

一个 2 TB gzip 可能无法从中间切开，即使 BE 很多也会形成单 Reader；适量的 Parquet/ORC 文件更容易形成并行 Scan Task。过少大文件和过多 KB 小文件都会损害效率。

得分关键词

文件数；可切分；BE；远端带宽；Tablet；小文件

常见误区：把并行度理解为一个可以无限调大的配置值。物理输入布局是硬上限。

7. Routine Load 中 Job、Task、Transaction 和 Offset 分别是什么？

Job 管长期生命周期，Task 执行一次微批，Transaction 提供原子写入，Offset 记录消费进度。

常驻 Job 持有 Kafka 配置和并发策略，不断生成短 Task。每个 Task 消费若干 Partition 范围并形成事务；事务成功后才推进对应 Offset，再生成下一轮 Task。

得分关键词

常驻 Job；短 Task；微批事务；消费进度

常见误区：把 Job 当成单个永不结束的事务；实际是连续的小事务序列。

8. 为什么 desired_concurrent_number 不等于实际消费并发？

因为实际 Task 数还受 Kafka Partition 数和 Doris 系统 Task 上限约束。

近似关系是 $\min(\text{Partition 数}, \text{desired}, \text{系统上限})$ 。Topic 只有 3 个 Partition 时最多只有 3 路独立消费；继续提高 desired 不能突破 Kafka 的并行切片数。

得分关键词

min；Kafka Partition；desired；系统上限

常见误区：看到 Lag 就直接把 desired 从 10 改成 100，忽略 Partition 和 BE 已饱和。

9. 三个 max_batch 参数如何共同决定微批次？

时间、行数、字节三个条件任意一个先达到，当前 Task 就结束并提交。

调小参数使数据更快可见，但事务、版本和小 Rowset 增多；调大提高批处理效率，却增加可见延迟和失败重试范围。应根据新鲜度 SLO 与长期 Compaction 能力折中。

得分关键词 interval ; rows ; size ; 任一先到 ; 吞吐延迟折中

常见误区：认为三个阈值必须同时满足才提交，或只调延迟而不评估版本数量。

10. Routine Load 怎样把 Kafka Offset 与 Doris 事务绑定，实现 Exactly-Once ？

只有微批事务成功提交后才持久化下一 Offset；事务失败则 Offset 不推进并重读同一范围。

例如消费 100-199 写入 T1，T1 Commit 后记录 200。若 T1 失败，从 100 重试；FE 切换时事务状态和进度由元数据恢复。这样避免先推进进度导致丢失，也避免已提交批次永久重复提交。

得分关键词 Offset ; 事务提交 ; 原子推进 ; 重试 ; FE 元数据

常见误区：只说 Kafka 有 Offset 就能 Exactly-Once；关键是 Offset 与目标事务提交必须绑定。

11. Routine Load 的 PAUSED 和 STOPPED 有什么区别？

PAUSED 是可恢复暂停，处理根因后可 RESUME；STOPPED 表示作业生命周期已结束。

认证、Offset、质量错误可能使作业进入 PAUSED。必须先看 ReasonOfStateChanged，否则反复 RESUME 会再次读到同一错误。STOP 通常用于永久结束，不应当作临时暂停。

得分关键词 PAUSED ; RESUME ; STOPPED ; Reason ; 根因

常见误区：把 RESUME 当成修复按钮，忽略脏数据或认证问题会导致暂停循环。

12. Flink Doris Connector 为什么仍然依赖 Stream Load ？

Connector 负责在 Flink Task 内缓冲和协调，但真正把批次写入 Doris 的通道仍是 Stream Load。

记录先进入 Connector Buffer，形成批次后通过 Stream Load 写入。因而批次大小、并行度、Label、事务、Tablet 分布和 Compaction 仍然决定性能；Flink 并不是逐行 JDBC INSERT。

得分关键词 Connector Buffer ; Stream Load ; 批次 ; Label ; Tablet

常见误区：认为使用 Flink 后 Doris 的导入链路和小批成本都不存在。

13. Flink Checkpoint 与 Doris Stream Load 2PC 怎样配合？

Flink Task 先把当前 Checkpoint 数据 PreCommit，只有全局 Checkpoint 成功后才通知 Doris Commit；失败则 Abort。

PreCommit 时数据已准备但不可见，Task 状态可被 Checkpoint 保存。全局成功后发布版本；若作业恢复，协调者根据已保存状态提交或放弃对应事务，从而避免旧 Checkpoint 重放造成重复可见。

得分关键词

Checkpoint；2PC；PreCommit；Commit；Abort；不可见

常见误区：把写入成功等同于 Checkpoint 成功；两者之间正是需要 2PC 协调的窗口。

14. 2PC 与 Sequence Column 分别解决什么问题？

2PC 解决跨系统故障重放与提交一致性；Sequence Column 解决同一主键业务事件乱序。

Checkpoint 成功前不发布，避免重放重复；但如果 SHIPPED 先到、旧 PAID 后到，两个事件都可能是合法事务，2PC 不知道谁更新。Sequence 根据业务版本或时间让较新事件获胜。

得分关键词

故障重放；提交边界；业务乱序；主键版本

常见误区：认为 Exactly-Once 自动保证事件按业务时间有序；传输一次与业务新旧是不同维度。

15. Group Commit 为什么能同时降低 FE 压力和 Compaction 压力？

它把多个小请求合成较少的大事务，既减少建事务和发布次数，也减少小版本与小 Rowset。

高频微批每次都支付 FE 解析/事务固定成本，并在热点 Tablet 产生新版本。BE 侧合并后，一批数据只需一次提交和更少 Flush，后台 Compaction 要归并的对象也明显减少。

得分关键词

服务器端攒批；事务数；版本；小 Rowset；Compaction

常见误区：把 Group Commit 只理解成网络压缩；主要收益来自事务和存储对象合并。

16. sync_mode 与 async_mode 的返回时机和可见性有何区别？

sync_mode 在合并事务提交并可见后返回；async_mode 写入 BE WAL 后较快返回，数据稍后可见。

sync 适合调用方要求返回后立即查询；async 用 WAL 保护尚未正式提交的数据，以更低响应延迟换取短暂不可见窗口。两者都不是免除失败处理和容量治理。

得分关键词

sync；async；WAL；返回时机；可见性

常见误区：认为 async 返回 Success 就表示查询立刻能读到数据。

17. Filtered Rows 和 Unselected Rows 有什么区别？

Filtered 是数据质量异常被丢弃；Unselected 是业务 WHERE 等规则主动排除，通常不计入错误率。

类型转换失败、超长、NULL 或无目标分区属于 Filtered；status='TEST' 被 WHERE 排除属于 Unselected。
max_filter_ratio 的核心分母是 Loaded + Filtered，不应把业务过滤误判为脏数据。

得分关键词

质量异常；业务过滤；max_filter_ratio；错误率

常见误区：只看总行数与 Loaded 差值，不区分是质量事故还是预期规则。

18. Label、事务原子性、幂等和 Exactly-Once 分别解决什么问题？

事务保证整批原子可见，Label 识别同批重试，幂等避免重复效果，Exactly-Once 还要求失败后不丢失。

固定 Label 让同一批最多成功一次，近似批次 At-Most-Once；上游还必须 At-Least-Once 重试，才能既不重又不丢。
Label 有作用域和保留周期；外部全局提交还可能需 2PC。

得分关键词

原子性；Label；幂等；At-Least-Once；Exactly-Once；保留期

常见误区：认为随机 UUID 就能幂等；如果重试重新生成 UUID，Doris 会认为是新批次。

19. Rowset、Segment、Cumulative Compaction 和 Base Compaction 是什么关系？

一次 Tablet 写入形成一个 Rowset，Rowset 包含若干 Segment；Cumulative 合并新小 Rowset，Base 再把累计结果并入历史基线。

MemTable Flush 生成列式 Segment，同一事务版本的 Segment 组成 Rowset。追加版本有利于顺序写，但会增加读放大。Cumulative 高频控制增量版本，Base 低频处理更大历史范围，二者共同平衡读写放大。

得分关键词

MemTable；Segment；Rowset；版本；Cumulative；Base

常见误区：把 Segment Compaction、Cumulative 和 Base 当作同一粒度；前者主要处理单 Rowset 内 Segment。

20. 如果导入正常但查询越来越慢，应按照什么顺序排查？

先从上游事务与小批产生速度查起，再看 Tablet/Rowset/版本和 Compaction，最后才调后台线程。

依次检查每分钟事务数、批次行数/字节、是否启用 Group Commit、单批跨分区数、桶数与热点、Rowset 数、Compaction Score、磁盘 IO 和查询写入争抢。若源头每秒制造大量小版本，只增加 Compaction 并发会放大资源竞争。

得分关键词

事务频率；批次；分区；热点 Tablet；Rowset；Compaction Score；IO

常见误区：看到查询慢就先改 SQL，或看到 Compaction 高就直接加线程，未追踪写入因果链。

综合自测标准

完成第七章后，应能给任意数据源选择导入方式，画出事务和进度边界，并从请求结果一路追踪到 MemTable、Rowset、版本与 Compaction。

能力层级	合格表现	优秀表现
选型	知道每种导入方式用途	能按方向、批量和事务边界解释选择
一致性	会使用 Label	能区分原子性、幂等、2PC、Offset 和 Sequence
性能	会调批次和并发	能解释小 Rowset、写放大与读放大
质量	会看 FilteredRows	能设计错误隔离、回放与质量监控
诊断	会看 Load 状态	能从阶段耗时、Lag、版本和 IO 串联根因

参考资料

- [1] Doris 3.x Load Overview
- [2] Doris 3.0 Stream Load
- [3] Doris 3.x Transactions
- [4] Doris 3.x Group Commit
- [5] Release 3.0.8